

SportLLM

Conception d'un Agent Conversationnel Intelligent
pour la Filière Équestre

Architecture RAG, Fine-Tuning et Mémoire Conversationnelle

Rapport de Projet de Fin d'Études

Projet de Recherche FP2ENG–25101-20260204

Jawad ARAB, Kenzi CHABI, Sofian EL YAMANI,
Amine GAHBICHE, Nassim ISSAD, Marwane KHALIS

Référente : Noama ADRA (Doctorante en IA)

Mentor : Serge PASSOLUNGHI

Efrei Paris – Février 2026

Résumé

Ce projet explore l'intégration de modèles de langage à grande échelle (LLM) dans le domaine de la compétition équestre. L'objectif est de pallier la complexité d'accès aux réglementations internationales (FEI) et nationales (FFE) via un chatbot intelligent. Nous avons implémenté une architecture hybride combinant Fine-Tuning (LoRA) pour l'ajustement du ton et RAG (Retrieval-Augmented Generation) pour la précision factuelle. L'utilisation de techniques avancées telles que le Multi-Query et le Reciprocal Rank Fusion (RRF) a permis une amélioration de la pertinence de 75.7% sur le Recall@3 tout en limitant les hallucinations. Un système de mémoire hybride (court terme et long terme) assure la continuité et la personnalisation des interactions. Ce rapport détaille la démarche scientifique, l'architecture technique et les résultats obtenus.

Mots-clés : Chatbot équestre, LLM, RAG, Fine-tuning, Mistral 7B, QLoRA, FEI, FFE, Mémoire conversationnelle.

Table des matières

Introduction Générale	5
1 Problématique et Cadre du Projet	6
1.1 Problématique	6
1.2 Sources Documentaires	6
1.3 Glossaire Technique	7
1.4 État de l'Art	7
1.5 Contributions des Acteurs du Projet	7
1.6 Hypothèses et Cadre Théorique	8
2 Stratégie d'Adaptation du Modèle : Le Fine-Tuning Supervisé (SFT)	8
2.1 Justification de l'approche et choix du modèle	8
2.1.1 Benchmark et sélection du modèle fondation	8
2.2 Constitution du Dataset d'Entraînement	9
2.2.1 Structure des données	9
2.2.2 Prétraitement et Nettoyage	10
2.3 Méthodologie Technique : QLoRA et PEFT	10
2.3.1 Le principe de LoRA	10
2.3.2 Configuration des Hyperparamètres	10
2.4 Protocole d'Évaluation : "LLM-as-a-Judge"	11
2.4.1 Méthodologie	11
2.4.2 Critères de Notation et Résultats	11
2.5 Limites du Fine-Tuning et Transition vers le RAG	12
3 Architecture RAG : Génération Augmentée par Récupération	12

3.1	Pourquoi le RAG ?	12
3.2	Architecture Globale du Système	13
3.3	Processus d'Ingestion et d'Indexation des Données	13
3.3.1	Le "Chunking" Stratégique	13
3.3.2	Encodage Vectoriel	13
3.3.3	Base de Données Vectorielle et Similarité Cosinus	13
3.4	Le Processus de Recherche (Retrieval)	14
3.5	Gestion de la Conversation : Le Module History Aware (HA)	14
3.5.1	La Problématique des Conversations Multi-Tours	14
3.5.2	Fonctionnement du Module History Aware	15
4	Amélioration du Pipeline RAG : Filtrage, Diversification et Fusion des Résultats	15
4.1	Limites de l'Approche RAG Naïve	15
4.2	Introduction d'un Seuil de Similarité (Threshold)	16
4.3	Diversification des Requêtes : Approche Multi-Query	17
4.4	Fusion et Re-classement des Résultats : Reciprocal Rank Fusion	17
4.5	Bénéfices Globaux pour le Chatbot Équestre	18
5	Système de Mémoire Conversationnelle	19
5.1	Introduction et Problématique	19
5.2	Mémoire Court Terme : Le Trimming Contextuel	19
5.2.1	Principe et Architecture	19
5.2.2	Implémentation Algorithmique	20
5.2.3	Résultats Expérimentaux	20
5.3	Mémoire Long Terme : Profil Utilisateur Vectorisé	21
5.3.1	Motivation et Cas d'Usage	21

5.3.2	Architecture Technique	21
5.3.3	Mécanisme d'Extraction et de Stockage	21
5.3.4	Injection dans le Prompt	22
5.3.5	Évaluation Qualitative	22
5.4	Intégration des Deux Systèmes	22
5.4.1	Flux de Décision Unifié	22
5.4.2	Gestion des Conflits	23
6	Évaluation et Métriques	23
6.1	Introduction à l'Évaluation	23
6.2	Les Métriques de Retrieval	24
6.2.1	Precision et Recall	24
6.2.2	Hit Rate et MRR	25
6.2.3	NDCG	25
6.3	Les Métriques LLM-as-Judge	26
6.4	Résultats Comparatifs	26
6.5	Analyse des Gains	27
6.6	Analyse des Compromis	27
	Conclusion Générale	28

Introduction Générale

Le sport équestre de haut niveau repose sur un cadre réglementaire d'une densité remarquable. La Fédération Française d'Équitation (FFE) et la Fédération Équestre Internationale (FEI) édictent chaque année des centaines de règles couvrant les dimensions des obstacles, les équipements autorisés, les protocoles de sécurité, les barèmes de notation et les procédures antidopage. Pour les cavaliers, entraîneurs, officiels et vétérinaires, l'accès rapide et fiable à cette information constitue un enjeu majeur : consulter des documents PDF de plusieurs centaines de pages lors d'un concours est inefficace, et une erreur d'interprétation peut entraîner l'élimination d'un concurrent ou mettre en danger la santé d'un cheval.

Dans ce contexte, le projet SportLLM propose de repenser l'accès à l'information réglementaire équestre en concevant un assistant conversationnel intelligent, capable de fournir des réponses d'une précision chirurgicale sur des points de réglementation complexes tout en maintenant une interaction fluide et naturelle. L'ambition ne se limite pas à un simple moteur de recherche amélioré : nous visons la création d'un véritable coach digital, adoptant une posture professionnelle et bienveillante adaptée aux attentes des acteurs de la filière.

La réalisation de cet objectif a nécessité de relever plusieurs défis techniques majeurs. Premièrement, les modèles de langage généralistes présentent des risques d'hallucination incompatibles avec un domaine où l'exactitude réglementaire est primordiale : dans un cadre compétitif, une information erronée sur un équipement peut mener à une élimination. Deuxièmement, ces modèles adoptent généralement un ton formel et encyclopédique, inadapté à l'interaction de coaching souhaitée. Troisièmement, les règlements évoluent chaque année, rendant obsolètes les connaissances figées lors de l'entraînement d'un modèle. Enfin, une expérience utilisateur fluide exige que le système maintienne le contexte conversationnel sur plusieurs échanges successifs et se souvienne du profil de chaque utilisateur.

Pour répondre à ces défis, nous avons conçu une architecture hybride combinant plusieurs techniques complémentaires. Le Fine-Tuning supervisé (QLoRA) permet d'adapter le comportement et le style du modèle Mistral 7B au domaine équestre. L'architecture RAG (Retrieval-Augmented Generation) connecte le modèle à une base documentaire actualisable, garantissant l'exactitude des informations. Des optimisations avancées du pipeline de recherche — incluant le filtrage par seuil de similarité, la diversification Multi-Query et la fusion par Reciprocal Rank Fusion — renforcent la fiabilité du système. Un système de mémoire hybride, combinant mémoire court terme (trimming contextuel) et mémoire long terme (profil utilisateur vectorisé), assure la continuité et la personnalisation

des interactions. Enfin, un framework d'évaluation rigoureux, mêlant métriques de retrieval et évaluation LLM-as-Judge, permet de quantifier objectivement les performances.

Ce rapport présente l'ensemble de ces contributions techniques. La première section expose la problématique et le cadre du projet. La deuxième section détaille la stratégie de fine-tuning du modèle. La troisième section présente l'architecture RAG de base et le module History Aware. La quatrième section décrit les améliorations apportées au pipeline de recherche. La cinquième section traite du système de mémoire conversationnelle. La sixième section présente le framework d'évaluation et les résultats comparatifs obtenus.

1 Problématique et Cadre du Projet

1.1 Problématique

Le sport équestre de haut niveau est régi par des cadres réglementaires denses, techniques et fréquemment mis à jour. Les acteurs de la filière (cavaliers, entraîneurs, vétérinaires) font face à un défi d'accessibilité à l'information. Consulter des documents PDF de plusieurs centaines de pages lors d'un concours est inefficace.

La problématique centrale de notre recherche est la suivante : *Comment concevoir un agent conversationnel capable de fournir des réponses d'une précision chirurgicale sur des points de réglementation complexes tout en maintenant une interaction fluide et naturelle ?*

Les enjeux identifiés sont triples. La véracité constitue le premier enjeu : dans un cadre compétitif, une erreur de l'IA (hallucination) sur un équipement peut mener à une élimination. Le contexte représente le deuxième enjeu : il est essentiel de différencier les règles FEI (international) des règles FFE (national). Enfin, la maintenabilité forme le troisième enjeu : il faut pouvoir mettre à jour les connaissances sans réentraînement complet du modèle.

1.2 Sources Documentaires

Le corpus de données a été constitué à partir de sources institutionnelles reconnues. La Fédération Équestre Internationale (FEI) a fourni les règlements de Saut d'Obstacles (Jumping), de Dressage et les listes de contrôle antidopage. La Fédération Française d'Équitation (FFE) a contribué avec le guide des épreuves Amateur et Pro. L'IFCE (Équipédia) a apporté sa base de connaissances technique sur la zootechnie et la santé équine.

1.3 Glossaire Technique

Afin de faciliter la lecture de ce rapport, nous définissons ici les termes techniques clés utilisés tout au long du document.

Le **RAG (Retrieval-Augmented Generation)** désigne une architecture permettant d'injecter des documents externes dans le prompt du LLM afin de fonder ses réponses sur des sources vérifiables. Le **Fine-tuning (LoRA)** correspond à l'ajustement partiel des paramètres du modèle pour spécialiser son vocabulaire et son comportement. Les **Embeddings** sont des représentations vectorielles du texte permettant le calcul de similarité sémantique entre concepts. Le **Vector Store** désigne une base de données (de type ChromaDB) stockant les vecteurs pour la recherche documentaire.

1.4 État de l'Art

L'état de l'art actuel montre une domination des architectures RAG pour les domaines où la donnée change rapidement. Ces architectures permettent de combiner la puissance générative des LLM avec la précision de sources documentaires externes, répondant ainsi au double impératif de fluidité conversationnelle et d'exactitude factuelle.

Nous avons testé plusieurs modèles de base pour identifier le candidat le plus adapté à notre cas d'usage. Le tableau 1 présente les résultats de ce benchmark préliminaire.

Modèle	Français	Précision Technique	Vitesse
Mistral 7B	Excellente	Haute	Moyenne
DeepSeek	Moyenne	Moyenne	Rapide
Llama 3	Bonne	Haute	Moyenne

TABLE 1 – Benchmark des LLM open-source testés

Notre choix s'est porté sur **Mistral 7B Instruct**, car il présente la meilleure compréhension des nuances de la langue française, indispensable pour interpréter des règlements juridico-sportifs.

1.5 Contributions des Acteurs du Projet

L'équipe est structurée pour couvrir l'ensemble du cycle de vie du projet. Nassim ISSAD, Sofian EL YAMANI et Kenzi CHABI ont travaillé sur la modélisation et le tuning, incluant l'implémentation du moteur de Q/A et le fine-tuning LoRA. Jawad ARAB et

Amine GAHBICHE ont piloté la recherche et la qualité, assurant les benchmarks et l'évaluation des hallucinations. Marwane KHALIS a assuré le développement et l'intégration, garantissant la cohérence entre le modèle fine-tuné et l'interface utilisateur.

1.6 Hypothèses et Cadre Théorique

Notre approche repose sur l'hypothèse qu'une architecture hybride est nécessaire pour atteindre les objectifs fixés. Le Fine-tuning permet d'adopter le ton d'un coach équestre professionnel, tandis que le RAG garantit l'exactitude des hauteurs d'obstacles, des équipements autorisés ou des pénalités applicables. Ces deux composantes, combinées à un système de mémoire conversationnelle, forment le socle technique de SportLLM.

2 Stratégie d'Adaptation du Modèle : Le Fine-Tuning Supervisé (SFT)

2.1 Justification de l'approche et choix du modèle

Dans le cadre du projet SportLLM, l'objectif premier était de fournir une assistance conversationnelle experte dans le domaine équestre de haut niveau (CSO, Dressage, CCE). Les modèles de langage généralistes disponibles sur le marché, bien que performants sur des tâches génériques, présentent deux lacunes majeures pour notre cas d'usage.

La première lacune concerne le manque de spécificité terminologique. Ces modèles peinent à utiliser le jargon précis de la Fédération Équestre Internationale (FEI). La seconde lacune porte sur la posture inadaptée : le ton est souvent trop formel, scolaire ou robotique, là où nos utilisateurs (cavaliers et coaches) attendent une interaction directe, un tutoiement professionnel et une autorité bienveillante.

Afin de pallier ces manques sans pour autant réentraîner un modèle depuis zéro — processus coûteux et énergivore — nous avons opté pour une stratégie de **Fine-Tuning (affinement)**.

2.1.1 Benchmark et sélection du modèle fondation

Nous avons comparé plusieurs modèles open-source de taille intermédiaire (7 à 8 milliards de paramètres) capables de tourner sur des infrastructures légères tout en offrant une bonne maîtrise du français.

TABLE 2 – Benchmark comparatif des modèles Open-Source

Modèle	Français	Raisonnement	Inférence	Verdict
DeepSeek	Moyenne	Haute	Rapide	Écarté (biais culturels)
Llama 3 (8B)	Haute	Très Haute	Moyenne	Écarté (Licence)
Mistral 7B v0.2	Excellente	Haute	Rapide	Sélectionné

Le choix s'est porté sur **Mistral 7B Instruct v0.2**. Ce modèle français présente l'avantage d'une tokenisation optimisée pour la langue française et d'une fenêtre de contexte de 32k tokens, idéale pour traiter des historiques de conversation longs.

2.2 Constitution du Dataset d'Entraînement

La qualité d'un fine-tuning dépend presque exclusivement de la qualité des données fournies ("*Garbage In, Garbage Out*"). Nous avons donc constitué un dataset propriétaire nommé `equestre_training_clean.jsonl`.

2.2.1 Structure des données

Le dataset est formaté en JSONL (JSON Lines). Chaque ligne représente une interaction complète structurée en trois rôles, format standard pour l'entraînement via la bibliothèque Hugging Face TRL :

1. **System Prompt** : Définit la persona (le Coach).
2. **User** : La question technique réelle.
3. **Assistant** : La réponse idéale (Vérité Terrain).

Listing 1 – Exemple d'une entrée du Dataset d'entraînement

```

1 {
2   "messages": [
3     {
4       "role": "system",
5       "content": "Tu es un coach equestre FEI olympique specialise en CSO."
6     },
7     {
8       "role": "user",
9       "content": "Comment utiliser un cardiofrequencemetre pour passer d'un CCI3* a un CCI4* ?"
10    },

```

```
11 {
12   "role": "assistant",
13   "content": "Trois indicateurs sont determinants. D'abord, mesurez
14     la reserve cardiaque : 240 moins la frequence moyenne..."
15 }
16 }
```

2.2.2 Prétraitement et Nettoyage

Le fichier brut contenait initialement 951 entrées. Une phase de nettoyage a été nécessaire pour supprimer les doublons, harmoniser la longueur des réponses et assurer la cohérence du tutoiement dans toutes les réponses “Assistant”.

2.3 Méthodologie Technique : QLoRA et PEFT

Pour adapter Mistral 7B sur nos ressources de calcul limitées (Google Colab T4 GPU avec 16 Go de VRAM), nous ne pouvions pas effectuer un *Full Fine-Tuning* qui aurait nécessité plus de 40 Go de mémoire. Nous avons utilisé la technique **PEFT (Parameter-Efficient Fine-Tuning)** couplée à **QLoRA (Quantized Low-Rank Adaptation)**.

2.3.1 Le principe de LoRA

Au lieu de modifier les 7 milliards de paramètres du modèle, LoRA gèle les poids du modèle pré-entraîné et injecte des matrices de rang inférieur (matrices A et B) dans les couches d'attention du Transformer. Ce sont uniquement ces matrices, représentant environ 1% à 2% des paramètres totaux, qui sont entraînées.

2.3.2 Configuration des Hyperparamètres

L'entraînement a été réalisé via la librairie `trl` et la classe `SFTTrainer`. Voici les hyperparamètres critiques validés lors de nos expérimentations :

- **Quantification (4-bit)** : Chargement du modèle en précision `nf4` (4-bit Normal Float) pour réduire l'empreinte mémoire par 4.
- **LoRA Rank (r) = 16** : Ce paramètre contrôle la complexité des nouvelles informations que le modèle peut apprendre. Un rang de 16 offre un bon compromis entre plasticité et risque de surapprentissage.

- **LoRA Alpha = 32** : Facteur d'échelle appliqué aux poids LoRA.
- **Cibles (Target Modules)** : Nous avons ciblé toutes les couches linéaires (`q_proj`, `k_proj`, `v_proj`, `o_proj`) pour maximiser l'impact sur le raisonnement.
- **Learning Rate = 2e-4** : Avec un scheduler de type "cosine" et une phase de chauffe (*warmup*) de 10 pas.
- **Époques = 3** : Suffisant pour converger sur un dataset de ~ 1000 lignes sans perdre les connaissances généralistes.

2.4 Protocole d'Évaluation : "LLM-as-a-Judge"

L'évaluation des modèles de langage génératifs est complexe car il n'existe pas de "bonne réponse" unique. Les métriques classiques (BLEU, ROUGE) sont inadaptées pour juger de la pertinence d'un conseil de coaching. Nous avons donc mis en place une pipeline d'évaluation automatisée dite "**LLM-as-a-Judge**".

2.4.1 Méthodologie

1. **Set de Test** : Isolation de 50 questions techniques jamais vues par le modèle durant l'entraînement.
2. **Génération Comparative** : Pour chaque question, nous générons deux réponses (Modèle A : Classique vs Modèle B : Fine-Tuné).
3. **Le Juge** : Un modèle supérieur analyse les deux réponses à l'aveugle par rapport à une réponse de référence.

2.4.2 Critères de Notation et Résultats

Le juge attribue une note de 0 à 10 selon deux axes définis dans le prompt d'évaluation : la **Posture Coach** et l'**Alignement Niche**.

TABLE 3 – Résultats de l'évaluation Fine-Tuning (Moyenne sur 50 échantillons)

Critère	Mistral Classique	Mistral Fine-Tuné	Gain
Posture / Ton	4.2 / 10	8.7 / 10	+4.5
Expertise Niche	5.1 / 10	8.2 / 10	+3.1

Interprétation Qualitative : Le modèle classique a tendance à rester très descriptif et encyclopédique. Le modèle fine-tuné adopte immédiatement la posture attendue : il a intégré le style conversationnel spécifique défini dans le dataset.

2.5 Limites du Fine-Tuning et Transition vers le RAG

Bien que le fine-tuning ait réussi à transformer le **comportement** et le **style** du modèle, nos tests ont révélé deux limitations intrinsèques à cette méthode :

- **L'Obsolescence des Connaissances** : Les règlements FEI changent chaque année. Le fine-tuning fige les connaissances à l'instant T de l'entraînement.
- **Les Hallucinations Factuelles** : Sur des questions très pointues (ex : barèmes spécifiques d'une épreuve locale), le modèle peut inventer des détails plausibles mais faux.

Conclusion partielle : Le Fine-Tuning est une excellente solution pour la *forme* mais insuffisante pour le *fond* (la véracité des données dynamiques). C'est ce constat qui a motivé l'hybridation de notre architecture avec un système **RAG**, détaillé dans la section suivante.

3 Architecture RAG : Génération Augmentée par Récupération

3.1 Pourquoi le RAG ?

Les Modèles de Langage de Grande Taille (LLM) comme GPT ou Claude possèdent une base de connaissances figée à leur date d'entraînement. De plus, ils sont incapables de citer avec précision des sources spécifiques qu'ils n'ont pas ingérées de manière exhaustive. Le RAG permet de contourner ces limites en connectant le LLM à une base de données documentaire externe.

L'avantage est double :

1. **Exactitude** : Les réponses sont basées sur des documents sources fournis (PDF officiels).
2. **Citations** : Le système peut théoriquement indiquer quel document a été utilisé pour répondre, renforçant la confiance de l'utilisateur.

Dans le contexte équestre, la fiabilité de l'information est primordiale. Un modèle de langage classique, bien qu'éloquent, présente des risques d'hallucination qui pourraient s'avérer dangereux pour la santé du cheval ou la conformité d'un cavalier en compétition.

3.2 Architecture Globale du Système

L'architecture SportLLM fonctionne comme un système hybride où les rôles sont clairement définis :

- **Le LLM (Moteur de Synthèse)** : Il traite les informations récupérées pour formuler une réponse cohérente en langage naturel.
- **Le Système de Recherche (Retrieval)** : Il agit comme un bibliothécaire numérique. Sa mission est de parcourir des milliers de lignes de texte en quelques millisecondes pour identifier les paragraphes les plus pertinents par rapport à la question posée.

3.3 Processus d'Ingestion et d'Indexation des Données

La qualité d'un RAG dépend principalement de la préparation des données en amont. Cette phase transforme des PDF statiques en une base de données "intelligente".

3.3.1 Le "Chunking" Stratégique

Le contenu des règlements PDF est trop massif pour être analysé d'un bloc. Nous avons donc fragmenté les textes en "chunks" (segments). Pour SportLLM, nous avons opté pour une taille de segment fixe avec un **overlap** (recouvrement) de 15%.

3.3.2 Encodage Vectoriel

Chaque segment est transformé en un vecteur mathématique par un encodeur. Le texte brut est passé à travers un modèle d'encodage qui transforme chaque mot ou phrase en un vecteur, c'est-à-dire une liste de coordonnées numériques dans un espace multidimensionnel. Cet espace est conçu de telle sorte que la proximité mathématique entre deux vecteurs reflète une proximité sémantique. Plus deux idées sont proches dans le sens (par exemple "selle" et "équipement"), plus leurs vecteurs seront proches géographiquement dans cet espace. Cette étape est cruciale car elle permet au système de s'affranchir d'une recherche par mots-clés rigide pour passer à une compréhension de l'intention de l'utilisateur.

3.3.3 Base de Données Vectorielle et Similarité Cosinus

Ces vecteurs sont stockés dans une base de données vectorielle. Lorsqu'une question est posée, elle est convertie en vecteur, et le système calcule l'angle entre le vecteur de la

question et ceux des documents (similarité cosinus). Les segments dont l'angle est le plus réduit sont considérés comme les plus proches sémantiquement et sont extraits pour la réponse.

3.4 Le Processus de Recherche (Retrieval)

La phase de récupération est le pont entre la question de l'utilisateur et la base de données. Pour une question telle que : *“Quelles sont les dimensions d'un obstacle de type vertical en club 2 ?”*, le système suit trois étapes :

1. **Vectorisation de la requête** : La question devient un point dans l'espace à 384 dimensions.
2. **Recherche de voisinage** : Le système identifie les chunks de texte les plus proches de ce point.
3. **Augmentation du prompt** : Les textes extraits sont envoyés au LLM avec la consigne : *“En utilisant uniquement ces informations, réponds à l'utilisateur.”*

3.5 Gestion de la Conversation : Le Module History Aware (HA)

L'innovation majeure de notre travail porte sur la gestion de l'historique. Un RAG classique traite chaque question de manière isolée, ce qui rend la conversation saccadée et frustrante.

3.5.1 La Problématique des Conversations Multi-Tours

Dans un échange réel, l'utilisateur fait référence à ses messages précédents sans les répéter :

— *Message 1* : “Comment se passe une compétition de saut d'obstacles ?”

(Plus tard dans la conversation)

— *Message 2* : “Y a-t-il des règles particulières ?”

Sans le module HA, le système chercherait le vecteur de la phrase “Y a-t-il des règles particulières ?”. Cette requête est trop générique : aucun document technique ne correspondra spécifiquement à cette phrase courte et vide de sujet.

3.5.2 Fonctionnement du Module History Aware

Le module History Aware agit comme une couche de reformulation intelligente. Avant de lancer la recherche, le système analyse l'historique et génère une **requête autonome**.

Exemple de reformulation History Aware

Entrée utilisateur : “Y a-t-il des règles particulières ?”

Historique détecté : La conversation porte sur le saut d'obstacles.

Traitement HA : Le module reformule en : “*Quelles sont les règles particulières concernant les compétitions de saut d'obstacles ?*”.

Résultat : Cette nouvelle question, riche en mots-clés sémantiques, permet une récupération pertinente des données dans les PDF sources.

4 Amélioration du Pipeline RAG : Filtrage, Diversification et Fusion des Résultats

Dans un système de Retrieval-Augmented Generation (RAG), la qualité de la réponse générée par le modèle de langage dépend directement de la pertinence des documents récupérés lors de la phase de recherche. Un pipeline RAG mal conçu peut rapidement devenir contre-productif : même avec un modèle performant, une récupération bruitée ou non pertinente peut conduire à des réponses incorrectes, approximatives, voire à des hallucinations.

Dans cette section, nous présentons les améliorations apportées à notre architecture RAG afin d'en renforcer la fiabilité, la précision et la robustesse dans un contexte métier exigeant. Ces améliorations reposent sur trois axes principaux : l'introduction d'un seuil de similarité (threshold) pour filtrer les documents non pertinents, la diversification des requêtes via une approche Multi-Query, et la fusion et le re-classement des résultats à l'aide de l'algorithme Reciprocal Rank Fusion (RRF).

4.1 Limites de l'Approche RAG Naïve

Dans sa forme la plus simple, un RAG fonctionne selon le principe suivant : pour une question utilisateur donnée, le système récupère les k documents les plus proches sémantiquement dans une base vectorielle, puis les transmet au LLM pour générer une réponse.

Cette approche, bien que fonctionnelle, présente une limite majeure : le système est

contraint de retourner un nombre fixe de documents, même lorsque ceux-ci sont peu ou pas pertinents. Autrement dit, l'absence d'information pertinente dans la base de connaissances n'empêche pas le système de fournir malgré tout du contexte au modèle.

Prenons un exemple volontairement hors domaine :

“Comment soigner une conjonctivite chez un chat ?”

Si la base de connaissances est exclusivement composée de documents liés à l'équitation et à la réglementation sportive, le moteur de recherche vectoriel retournera malgré tout les documents les plus proches d'un point de vue mathématique, bien qu'ils soient sémantiquement hors sujet. Le LLM, recevant ce contexte non pertinent, risque alors de produire une réponse erronée ou hallucinée.

Ce comportement est particulièrement problématique dans un domaine réglementé et compétitif, où la précision des informations est essentielle.

4.2 Introduction d'un Seuil de Similarité (Threshold)

Pour remédier à ce problème, nous avons introduit un seuil minimal de similarité sémantique, appelé *threshold*. L'idée est simple : un document n'est considéré comme pertinent que si son score de similarité avec la requête dépasse une valeur définie.

Formellement, soit $sim(q, d)$ la similarité entre la requête q et un document d . Un document est sélectionné si :

$$sim(q, d) \geq \tau$$

où τ représente le seuil de similarité.

Dans notre implémentation, nous avons empiriquement fixé ce seuil à $\tau = 0,3$, valeur offrant un bon compromis entre rappel et précision dans notre corpus métier.

Reprenons l'exemple précédent sur la conjonctivite chez le chat. Aucun document de la base ne dépassant ce seuil, le système retourne alors un contexte vide. Le LLM est explicitement informé qu'aucune information fiable n'a été trouvée, ce qui permet d'éviter les hallucinations, d'inciter le modèle à exprimer une incertitude, et de préserver la crédibilité globale du chatbot.

Dans le contexte équestre, ce mécanisme est crucial pour empêcher le modèle de répondre à des questions hors périmètre, par exemple médicales ou vétérinaires, lorsque la base ne contient pas de sources adaptées.

4.3 Diversification des Requêtes : Approche Multi-Query

Même avec un seuil de similarité, une autre difficulté subsiste : l'utilisateur ne formule pas toujours sa question de manière optimale. Une requête unique peut manquer certains termes clés présents dans la base de connaissances, ce qui réduit la capacité du système à retrouver l'ensemble des documents pertinents.

Pour pallier ce problème, nous avons implémenté une approche dite *Multi-Query*. Le principe consiste à générer automatiquement plusieurs reformulations sémantiquement équivalentes de la question initiale, chacune explorant un angle différent du même besoin informationnel.

Par exemple, pour la question :

“Comment améliorer l'engagement de mon cheval ?”

Le système génère des variantes telles que :

- “Quels exercices permettent d'augmenter l'engagement du cheval ?”
- “Travail à effectuer pour améliorer l'équilibre et l'engagement du cheval”
- “Méthodes d'entraînement pour renforcer l'engagement en saut d'obstacles”

Chaque reformulation est ensuite utilisée comme une requête indépendante vers la base vectorielle. Cette stratégie permet d'augmenter le rappel (retrouver plus de documents pertinents), de réduire la dépendance à une formulation unique, et de mieux couvrir la diversité lexicale du corpus métier.

Dans un domaine technique comme l'équitation de compétition, où un même concept peut être exprimé de multiples façons (engagement, impulsion, propulsion, équilibre), cette approche s'avère particulièrement efficace.

4.4 Fusion et Re-classement des Résultats : Reciprocal Rank Fusion

L'approche Multi-Query génère toutefois un nouveau défi : chaque requête produit sa propre liste de documents classés par pertinence. Il est donc nécessaire de fusionner ces résultats de manière cohérente afin de sélectionner les documents les plus fiables.

Pour cela, nous avons utilisé l'algorithme *Reciprocal Rank Fusion* (RRF). Le RRF attribue à chaque document un score basé sur sa position dans les classements issus des différentes requêtes. Plus un document apparaît haut dans plusieurs listes, plus son score global augmente.

Le score RRF d'un document d est défini comme :

$$RRF(d) = \sum_{i=1}^n \frac{1}{k + rank_i(d)}$$

où :

- $rank_i(d)$ est le rang du document d dans la requête i ,
- k est une constante (généralement fixée à 60) pour limiter l'influence des rangs très élevés.

Cette méthode présente plusieurs avantages : elle favorise les documents consensuels, elle est robuste aux requêtes bruitées, et elle ne dépend pas des scores de similarité absolus mais uniquement du classement relatif.

Dans notre cas, si un document traitant des hauteurs réglementaires en Amateur 1 apparaît systématiquement dans les premiers résultats de plusieurs reformulations, le RRF le propulse naturellement en tête. Le LLM reçoit ainsi un contexte plus fiable et mieux validé.

4.5 Bénéfices Globaux pour le Chatbot Équestre

La combinaison du seuil de similarité, du Multi-Query et du RRF permet de transformer un RAG classique en un système robuste et expert. Le chatbot ne se contente plus de récupérer des informations approximatives, mais sélectionne activement les sources les plus pertinentes et les mieux documentées.

Cette rigueur dans la phase de retrieval se traduit directement par une réduction significative des hallucinations, une amélioration de la précision technique des réponses, une meilleure confiance accordée par les utilisateurs, et une adéquation accrue avec les exigences du domaine compétitif équestre.

Ainsi, le chatbot dépasse le simple rôle conversationnel pour devenir un véritable assistant de coaching, capable de fournir des recommandations fiables, contextualisées et exploitables.

5 Système de Mémoire Conversationnelle

5.1 Introduction et Problématique

L'un des défis majeurs dans la conception d'un agent conversationnel intelligent réside dans sa capacité à maintenir une cohérence contextuelle tout au long d'un échange multi-tours. Dans le cadre de SportLLM, cette problématique est d'autant plus critique que les cavaliers posent souvent des questions en cascade, où chaque nouvelle question s'appuie implicitement sur les échanges précédents.

Par exemple, un utilisateur peut demander : “*Quelle est la hauteur maximale pour un mors de bride en CSO Amateur 1 ?*”, puis enchaîner avec “*Et pour l'Amateur Elite ?*” ou encore “*Ce type de mors est-il autorisé avec un levier ?*”. Sans mécanisme de mémoire, le chatbot traiterait chaque question de manière isolée, perdant ainsi le fil conducteur de la conversation.

Notre architecture propose une solution hybride en deux couches :

- **Mémoire Court Terme (Short-Term Memory)** : Conservation du contexte immédiat de la conversation via un mécanisme de *Trimming* optimisé.
- **Mémoire Long Terme (Long-Term Memory)** : Stockage vectoriel d'informations persistantes sur le profil utilisateur (discipline pratiquée, niveau de compétition, équipement habituel).

Cette approche s'inspire des travaux de *MemGPT* et des architectures à mémoire épisodique utilisées dans les systèmes de dialogue avancés.

5.2 Mémoire Court Terme : Le Trimming Contextuel

5.2.1 Principe et Architecture

La mémoire court terme repose sur le principe du **Context Window Management**. Chaque modèle de langage possède une fenêtre de contexte limitée (pour Mistral 7B : 8192 tokens). Au fur et à mesure d'une conversation, l'historique s'accumule et risque de saturer cette fenêtre.

Le *Trimming* consiste à :

1. Conserver les N derniers tours de parole (typiquement $N = 5$)
2. Supprimer les échanges les plus anciens pour libérer de l'espace

3. Maintenir systématiquement le prompt système initial pour garantir le comportement du chatbot

5.2.2 Implémentation Algorithmique

L'algorithme de trimming a été implémenté avec une logique de priorité :

Algorithm 1 Trimming Contextuel

Require: Historique complet H , Seuil de tokens T_{max} , Nombre de tours N

Ensure: Contexte optimisé C

```

1:  $C \leftarrow \text{prompt\_système}$ 
2:  $tokens\_count \leftarrow \text{count\_tokens}(\text{prompt\_système})$ 
3:  $recent\_turns \leftarrow H[-N:]$  {Récupère les N derniers tours}
4: for each  $turn$  in  $recent\_turns$  (reversed) do
5:    $turn\_tokens \leftarrow \text{count\_tokens}(turn)$ 
6:   if  $tokens\_count + turn\_tokens \leq T_{max}$  then
7:      $C \leftarrow turn + C$ 
8:      $tokens\_count \leftarrow tokens\_count + turn\_tokens$ 
9:   else
10:    break {Arrêt si dépassement}
11:   end if
12: end for
13: return  $C$ 

```

Cette approche garantit que le contexte le plus récent est toujours préservé, que le prompt système (ton de coach équestre) reste actif, et que la fenêtre de contexte n'est jamais saturée.

5.2.3 Résultats Expérimentaux

Nous avons évalué l'impact du trimming sur la cohérence conversationnelle via un jeu de test de 50 conversations multi-tours (moyenne de 7 échanges par conversation).

Méthode	Cohérence contextuelle	Tokens moyens/réponse
Sans mémoire	42%	1200
Trimming ($N = 3$)	78%	2800
Trimming ($N = 5$)	91%	3500
Trimming ($N = 10$)	89%	5200

TABLE 4 – Impact du nombre de tours conservés sur la cohérence

L'optimum se situe à $N = 5$ tours, offrant le meilleur compromis entre cohérence et charge computationnelle. Au-delà, les gains deviennent marginaux tandis que la latence augmente de manière significative.

5.3 Mémoire Long Terme : Profil Utilisateur Vectorisé

5.3.1 Motivation et Cas d'Usage

La mémoire court terme résout la continuité immédiate, mais elle est volatile (réinitialisée à chaque nouvelle session). Or, un cavalier fidèle à l'outil peut revenir plusieurs jours plus tard et s'attendre à ce que le chatbot se souvienne de sa discipline (CSO, Dressage, CCE) ou de son niveau de compétition (Amateur 1, Pro Elite).

La mémoire long terme répond à trois objectifs :

1. **Personnalisation** : Adapter automatiquement les réponses au contexte de l'utilisateur
2. **Efficacité** : Éviter de redemander systématiquement "Quel est votre niveau ?"
3. **Recommandations proactives** : "Pour votre prochain CSO Amateur Elite, pensez à vérifier la conformité de votre nouvelle selle"

5.3.2 Architecture Technique

La mémoire long terme repose sur un système de *Vector Store* dédié, distinct de la base documentaire des règlements. Nous utilisons ChromaDB avec une collection spécifique `user_profiles`.

5.3.3 Mécanisme d'Extraction et de Stockage

L'alimentation de la mémoire long terme se fait en deux temps.

Phase 1 : Détection d'entités Un modèle NER (Named Entity Recognition) léger identifie dans la conversation les entités pertinentes :

- Discipline : "CSO", "Dressage", "Hunter"
- Niveau : "Amateur 1", "Pro Elite", "Preliminaire"
- Équipement : "Mors Pelham", "Selle Devoucoux"
- Cheval : "Nom, race, âge"

Phase 2 : Embedding et indexation Chaque information extraite est transformée en vecteur via `all-MiniLM-L6-v2`, stockée dans ChromaDB avec un identifiant utilisateur unique, et horodatée pour permettre l'évolution du profil.

Exemple de document stocké :

```
{
  "user_id": "uuid-12345",
  "entity_type": "discipline",
  "value": "CSO Amateur Elite",
  "timestamp": "2026-02-01T14:30:00Z",
  "embedding": [0.12, -0.08, 0.45, ...]
}
```

5.3.4 Injection dans le Prompt

Lors d'une nouvelle requête, le système encode la question de l'utilisateur, recherche dans la collection `user_profiles` les 3 entités les plus similaires, puis injecte un préambule dans le prompt :

“Tu es Coach Équestre, un assistant spécialisé. L'utilisateur pratique le CSO niveau Amateur Elite et possède un cheval de race Selle Français nommé Tornado. Adapte tes conseils en conséquence.”

5.3.5 Évaluation Qualitative

Nous avons mesuré l'impact de la mémoire long terme sur 30 sessions utilisateurs réelles (testeurs bêta de la FFE) :

Critère	Sans MLT	Avec MLT
Pertinence des recommandations	6.2/10	8.7/10
Nombre de questions de contexte posées	2.8	0.4
Satisfaction utilisateur (NPS)	+12	+34

TABLE 5 – Impact de la mémoire long terme sur l'expérience utilisateur

La mémoire long terme réduit drastiquement les allers-retours de clarification, rendant l'interaction plus fluide et professionnelle.

5.4 Intégration des Deux Systèmes

5.4.1 Flux de Décision Unifié

L'orchestration entre mémoire court terme et long terme suit une logique hiérarchique :

Algorithm 2 Génération de Réponse avec Mémoire Hybride

Require: Requête Q , User ID U **Ensure:** Réponse R

- 1: $context_{short} \leftarrow \text{get_trimmed_history}(U, N = 5)$
 - 2: $context_{long} \leftarrow \text{retrieve_user_profile}(U, top_k = 3)$
 - 3: $prompt \leftarrow \text{build_prompt}(context_{long}, context_{short}, Q)$
 - 4: $documents \leftarrow \text{RAG_retrieval}(Q)$ {Recherche documentaire}
 - 5: $R \leftarrow \text{LLM.generate}(prompt, documents)$
 - 6: $\text{update_short_memory}(U, Q, R)$ {Ajoute l'échange}
 - 7: $\text{extract_and_store_entities}(R) \rightarrow \text{update_long_memory}(U)$
 - 8: **return** R
-

Cette architecture garantit que :

- Les informations récentes (court terme) priment sur les informations anciennes (long terme)
- Le profil utilisateur s'enrichit automatiquement au fil des conversations
- Le système reste réactif (latence <15 secondes)

5.4.2 Gestion des Conflits

Un mécanisme de résolution de conflits a été implémenté. Si la mémoire long terme indique "Niveau : Amateur 1" mais que l'utilisateur vient de mentionner "Je viens de passer Pro Elite", le système :

1. Détecte l'incohérence temporelle
2. Demande une confirmation explicite : *"J'avais noté que vous étiez Amateur 1, dois-je mettre à jour votre profil en Pro Elite?"*
3. Met à jour la base vectorielle après validation

6 Évaluation et Métriques

6.1 Introduction à l'Évaluation

L'évaluation d'un système RAG représente un défi particulier car ce type de système combine deux composantes fondamentalement différentes : la récupération de documents (retrieval) et la génération de réponses. Ces deux composantes nécessitent des approches d'évaluation distinctes et complémentaires, c'est pourquoi nous avons structuré notre framework autour de deux familles de métriques.

Évaluer un système RAG ne se résume pas à vérifier si les réponses semblent correctes. Sans métriques rigoureuses, il est impossible de quantifier les améliorations, de comparer objectivement différentes configurations, ou d'identifier les points faibles du système. L'évaluation par métriques permet de passer d'une appréciation subjective à une mesure objective et reproductible.

Un système RAG peut échouer de deux manières distinctes. Premièrement, il peut ne pas retrouver les bons documents : dans ce cas, même le meilleur modèle de génération ne pourra produire une réponse correcte car il n'aura pas accès aux informations nécessaires. Deuxièmement, il peut retrouver les bons documents mais générer une mauvaise réponse contenant des hallucinations ou des informations mal interprétées. Chaque mode d'échec nécessite des métriques spécifiques pour être détecté et corrigé.

Pour calculer les métriques de retrieval, nous avons établi une vérité terrain (ground truth) par une méthode semi-automatique basée sur des mots-clés. Cette approche permet d'identifier quels documents du corpus contiennent effectivement les informations nécessaires pour répondre à chaque question de test.

6.2 Les Métriques de Retrieval

Les métriques de retrieval évaluent la première étape du pipeline RAG : la capacité du système à retrouver les documents pertinents dans le corpus. Ces métriques sont fondamentales car elles conditionnent directement la qualité de la réponse finale.

6.2.1 Precision et Recall

La **Precision** mesure la proportion de documents pertinents parmi ceux retournés par le système. Elle répond à la question : parmi tous les documents récupérés, combien sont réellement utiles ?

$$Precision@K = \frac{|\text{Documents pertinents dans les K premiers}|}{K} \quad (1)$$

Le **Recall** mesure la proportion de documents pertinents du corpus qui ont été effectivement retrouvés. Il répond à une question différente : parmi tous les documents pertinents qui existent, combien avons-nous réussi à récupérer ?

$$Recall@K = \frac{|\text{Documents pertinents retrouvés}|}{|\text{Total documents pertinents}|} \quad (2)$$

Il existe un compromis inhérent entre précision et recall. Augmenter le nombre de documents récupérés améliore généralement le recall mais tend à diminuer la précision. L'enjeu est de trouver le bon équilibre en fonction des priorités de l'application.

6.2.2 Hit Rate et MRR

Le **Hit Rate** offre une perspective complémentaire en répondant à une question binaire simple : le système a-t-il trouvé au moins un document pertinent pour cette requête ?

$$\text{Hit Rate} = \frac{|\text{Requêtes avec au moins 1 hit}|}{|\text{Total requêtes}|} \quad (3)$$

Le **Mean Reciprocal Rank (MRR)** introduit la notion de position dans l'évaluation. Il ne suffit pas de retrouver les documents pertinents, il faut également les placer en tête de liste :

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (4)$$

6.2.3 NDCG

Le **Normalized Discounted Cumulative Gain (NDCG)** est la métrique de classement la plus sophistiquée de notre framework. Elle prend en compte non seulement la présence de documents pertinents, mais aussi leur position relative dans le classement avec un discounting logarithmique :

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}} \quad \text{où} \quad \text{DCG} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)} \quad (5)$$

Le principe du NDCG repose sur deux idées. Les documents pertinents placés en tête de liste apportent plus de valeur que ceux placés en fin de liste. Le score obtenu est normalisé par rapport au score idéal qu'on obtiendrait si tous les documents pertinents étaient classés en premier.

6.3 Les Métriques LLM-as-Judge

Les métriques de retrieval, bien qu'essentielles, ne capturent pas la qualité de la réponse finale. Un système peut récupérer tous les bons documents mais produire une réponse confuse, incomplète ou contenant des hallucinations. C'est pourquoi nous avons implémenté une deuxième famille de métriques utilisant GPT-4o comme évaluateur automatique.

L'approche LLM-as-Judge consiste à utiliser un modèle de langage puissant pour évaluer la qualité des réponses selon des critères qualitatifs. Le modèle évaluateur reçoit la question originale, le contexte fourni, et la réponse générée, puis attribue un score selon des critères prédéfinis.

La **Faithfulness** (fidélité) mesure dans quelle mesure la réponse générée est fidèle aux documents sources, sans invention ni extrapolation. Un score de 0.96 est excellent car il signifie que dans 96% des cas, les réponses sont entièrement fidèles aux sources. Cette métrique est particulièrement importante dans un contexte réglementaire.

La **Relevance** (pertinence) évalue si la réponse répond effectivement à la question posée. Cette métrique capture un mode d'échec différent : une réponse peut être parfaitement fidèle aux sources mais complètement hors sujet si le retrieval a récupéré des documents non pertinents.

La **Context Precision** mesure la qualité des documents fournis au modèle de génération, du point de vue de leur utilité pour répondre à la question.

6.4 Résultats Comparatifs

L'évaluation a été conduite sur 19 questions de test couvrant différents aspects de la réglementation FEI. Le tableau ci-dessous synthétise les résultats obtenus pour chaque métrique :

Métrique	Classique	Amélioré	Évolution	Bilan
Recall@3	0.39	0.68	+75.7%	Gain majeur
Recall@5	0.68	0.75	+9.6%	Amélioration
Hit Rate	0.68	0.74	+7.7%	Amélioration
MRR	0.64	0.67	+3.8%	Amélioration
NDCG	0.65	0.69	+5.9%	Amélioration
Context Precision	0.54	0.67	+23.3%	Gain significatif
Relevance	0.63	0.63	0.0%	Stable
Precision@3	0.63	0.53	-16.7%	Dégradation
Faithfulness	0.96	0.92	-4.4%	Légère baisse

TABLE 6 – Comparaison des métriques entre RAG Classique et RAG Amélioré

Le bilan global montre que le RAG Amélioré surpasse le RAG Classique sur 6 métriques, reste égal sur 1 métrique, et performe moins bien sur 2 métriques. Ce résultat démontre l'efficacité des optimisations implémentées.

6.5 Analyse des Gains

Le gain le plus spectaculaire concerne le **Recall@3 avec une amélioration de 75.7%**. Cette métrique passe de 0.39 à 0.68, signifiant que le système amélioré retrouve désormais plus des deux tiers des documents pertinents contre moins de 40% auparavant. Cette amélioration considérable résulte de la stratégie Multi-Query qui génère plusieurs reformulations de la question, combinée à l'algorithme RRF qui fusionne intelligemment les résultats de ces différentes recherches.

La **Context Precision progresse de 23.3%**, passant de 0.54 à 0.67. Cette amélioration s'explique par le filtrage par seuil de score (threshold 0.3) qui élimine les documents de faible pertinence, produisant un contexte plus propre et plus ciblé pour le modèle de génération.

6.6 Analyse des Compromis

La **Precision@3 diminue de 16.7%**, passant de 0.63 à 0.53. Cette dégradation est une conséquence directe de la stratégie Multi-Query : en explorant plusieurs formulations de la question, le système récupère un ensemble plus large de documents, ce qui augmente mécaniquement le recall mais dilue la precision. Ce compromis est toutefois acceptable dans notre contexte car le coût de manquer une information pertinente (faux négatif) est généralement plus élevé que le coût d'inclure un document marginalement pertinent (faux

positif). Le modèle de génération peut filtrer les informations non pertinentes lors de la rédaction de sa réponse, mais il ne peut pas deviner des informations qu'il n'a pas reçues.

La **Faithfulness diminue légèrement de 4.4%**, passant de 0.96 à 0.92. Bien que cette baisse mérite attention, les deux scores restent excellents et bien au-dessus du seuil acceptable de 0.90. Un contexte plus riche peut offrir davantage d'opportunités d'interprétation, ce qui explique cette légère dégradation.

L'évaluation a également mesuré l'impact sur la latence. Le **temps moyen de réponse** passe de 13.0 secondes à 15.8 secondes, soit une augmentation de 21.2% ou 2.8 secondes supplémentaires. Cette latence additionnelle s'explique par les étapes supplémentaires du RAG Amélioré : génération des variantes de requêtes via Multi-Query et fusion des résultats via RRF. Pour une application où la qualité prime sur la vitesse, ce compromis est acceptable.

Conclusion Générale

Le projet SportLLM démontre qu'il est possible de construire un assistant conversationnel expert dans un domaine technique exigeant comme l'équitation de compétition. En combinant plusieurs techniques complémentaires au sein d'une architecture hybride, nous avons réussi à créer un système qui répond aux exigences de précision, de personnalisation et de fluidité conversationnelle attendues par les professionnels de la filière équestre.

Le Fine-Tuning supervisé avec QLoRA a permis de transformer le comportement du modèle Mistral 7B, lui conférant une posture de coach équestre authentique avec des gains significatifs sur les critères de ton (+4.5 points) et d'expertise niche (+3.1 points). Cette adaptation comportementale, réalisée en n'entraînant que 1 à 2% des paramètres du modèle, illustre l'efficacité des techniques PEFT modernes pour spécialiser un modèle généraliste à moindre coût.

L'architecture RAG a résolu le problème fondamental de l'obsolescence et de l'exactitude des connaissances. En connectant le modèle à une base documentaire actualisable des règlements FFE et FEI, nous garantissons que les réponses sont fondées sur des sources officielles et non sur des connaissances figées. Le module History Aware assure par ailleurs la fluidité des conversations multi-tours, permettant aux utilisateurs de s'exprimer naturellement sans avoir à reformuler systématiquement le contexte de leurs questions.

Les optimisations du pipeline de recherche — seuil de similarité, Multi-Query et Reciprocal Rank Fusion — ont considérablement renforcé la fiabilité du système. L'amélioration de 75.7% du Recall@3 et de 23.3% de la Context Precision témoigne de l'efficacité de

ces techniques pour produire un contexte de meilleure qualité, au prix d'un compromis acceptable sur la Precision@3 et d'une légère augmentation de la latence.

Le système de mémoire hybride, combinant trimming contextuel pour la mémoire court terme et stockage vectoriel pour la mémoire long terme, apporte une dimension de personnalisation essentielle. L'amélioration du NPS de +12 à +34 et la réduction des questions de clarification de 2.8 à 0.4 par session démontrent l'impact positif sur l'expérience utilisateur.

Enfin, le framework d'évaluation mis en place, articulant métriques de retrieval et évaluation LLM-as-Judge, constitue un outil méthodologique reproductible pour objectiver les performances et guider les optimisations futures. Les métriques transforment l'évaluation d'un exercice subjectif en une démarche scientifique comparable et itérative.

Plusieurs perspectives d'amélioration se dessinent pour les développements futurs. L'implémentation d'une **analyse multimodale** permettrait aux cavaliers de photographier leur équipement pour une vérification automatique de conformité. Une **mémoire sémantique** plus avancée pourrait capturer les préférences implicites des utilisateurs au-delà des entités nommées. Un mécanisme d'**oubli contrôlé** (decay) gérerait l'évolution naturelle des profils utilisateurs (changement de niveau, de cheval, etc.). Enfin, une **mémoire partagée** entre entraîneur et élèves ouvrirait de nouvelles possibilités d'usage collectif, faisant de SportLLM un véritable outil collaboratif pour les écuries.

Le projet SportLLM constitue ainsi une contribution significative au domaine des assistants conversationnels spécialisés, prouvant qu'une architecture hybride bien conçue peut atteindre un niveau de performance et de fiabilité compatible avec des usages professionnels exigeants dans le sport de haut niveau.

Références

- [1] Mistral AI (2023). *Announcing Mistral 7B*.
- [2] FEI. *Jumping Regulations and World Cup Rules 2024*.
- [3] LangChain AI (2024). *Advanced RAG Series : Multi-Query & RRF*.
- [4] Hu, E. J., et al. (2021). *LoRA : Low-Rank Adaptation of Large Language Models*. arXiv :2106.09685.
- [5] Dettmers, T., et al. (2023). *QLoRA : Efficient Finetuning of Quantized Language Models*. arXiv :2305.14314.
- [6] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
- [7] Packer, C., et al. (2023). *MemGPT : Towards LLMs as Operating Systems*. arXiv :2310.08560.
- [8] Cormack, G. V., Clarke, C. L. A., Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. SIGIR 2009.
- [9] Fédération Française d'Équitation. *Guide des Épreuves Amateur et Pro 2024-2025*.